

Metody v C#

Příprava k maturitě - SPŠT IT

Obsah

1 Pojem metoda	2
2 Deklarace a implementace metody	2
2.1 Syntaxe	2
2.2 Části metody	2
2.3 Příklady základních metod	2
2.4 Příkaz return	3
3 Volání metod	3
3.1 Statické metody	3
3.2 Instance metody (nestatické)	4
3.3 Různé způsoby volání	4
4 Formální a skutečné parametry	5
4.1 Formální parametry	5
4.2 Skutečné parametry (argumenty)	5
4.3 Pojmenované argumenty	5
4.4 Volitelné parametry	6
5 Modifikátory parametrů	6
5.1 Předávání hodnotou (výchozí)	6
5.2 Modifikátor ref	6
5.3 Modifikátor out	7
5.4 Modifikátor in (C# 7.2+)	7
5.5 Modifikátor params	8
5.6 Porovnání modifikátorů	8
6 Přetěžování metod (Method Overloading)	9
6.1 Pravidla přetěžování	9
6.2 Příklady přetěžování	9
6.3 Přetěžování s modifikátory	10
6.4 Praktický příklad	10
6.5 Výhody přetěžování	11
Odpovědi na zadané podotázky	11
Další materiály a zdroje	11

1 Pojem metoda

Metoda je pojmenovaný blok kódu, který provádí specifickou úlohu. V C# jsou metody vždy součástí třídy nebo struktury.

Metody v C# poskytují:

- **Znovupoužitelnost** – stejný kód lze volat opakovaně
- **Organizaci** – logické seskupení funkcionality
- **Zapouzdření** – skrytí implementačních detailů
- **Abstrakci** – zjednodušení složitých operací
- **Modularitu** – rozdělení programu na menší části

Rozdíl mezi metodou a funkcí:

- V C++ mluvíme o **funkcích**
- V C# mluvíme o **metodách** (jsou vždy součástí třídy)
- Princip je stejný, terminologie se liší

2 Deklarace a implementace metody

2.1 Syntaxe

CS

```
1 // Základní struktura metody
2 modifikatorPristupu navratovyTyp NazevMetody(typParametru1 parametr1,
  typParametru2 parametr2) {
3     // tělo metody
4     return hodnota; // pokud návratový typ není void
5 }
```

2.2 Části metody

1. **Modifikátor přístupu** – `public`, `private`, `protected`, `internal`
2. **Návratový typ** – typ hodnoty, kterou metoda vrátí (`int`, `string`, `void`...)
3. **Název metody** – identifikátor metody (PascalCase v C#)
4. **Seznam parametrů** – vstupy metody (může být prázdný)
5. **Tělo metody** – kód ohraničený složenými závorkami
6. **Příkaz return** – vrátí hodnotu volajícímu kódu

2.3 Příklady základních metod

CS

```
1 class Program {
2     // Metoda bez parametrů a bez návratové hodnoty
3     static void Pozdrav() {
4         Console.WriteLine("Ahoj!");
5     }
6
7     // Metoda s parametry a návratovou hodnotou
8     static int Soucet(int a, int b) {
9         return a + b;
10    }
11 }
```

```

12 // Metoda s více parametry
13 static double Objem(double delka, double sirka, double vyska) {
14     return delka * sirka * vyska;
15 }
16
17 // Metoda vracející boolean
18 static bool JeKladne(int cislo) {
19     return cislo > 0;
20 }
21
22 // Metoda vracející string
23 static string ZiskejPozdrav(string jmeno) {
24     return $"Ahoj, {jmeno}!";
25 }
26 }

```

2.4 Příkaz return

Příkaz `return` má dva účely:

1. **Ukončí vykonávání metody** – žádný další kód po `return` se neprovede
2. **Vrátí hodnotu** – pokud metoda není typu `void`

cs

```

1 static int AbsolutniHodnota(int cislo) {
2     if (cislo < 0) {
3         return -cislo; // ukončí metodu zde
4     }
5     return cislo; // jinak vrátí původní hodnotu
6 }
7
8 static void VypisHodnotu(int x) {
9     if (x < 0) {
10         Console.WriteLine("Záporné číslo");
11         return; // předčasné ukončení (void metoda)
12     }
13     Console.WriteLine($"Kladné číslo: {x}");
14 }

```

3 Volání metod

3.1 Statické metody

Statické metody se volají přes název třídy nebo přímo (pokud jsou ve stejné třídě):

cs

```

1 class Matematika {
2     static int Soucet(int a, int b) {
3         return a + b;
4     }
5
6     static void Main() {
7         // Volání metody ve stejné třídě

```

```

8         int vysledek = Soucet(5, 3);
9
10        // Volání metody z jiné třídy
11        int vysledek2 = Matematika.Soucet(10, 20);
12    }
13 }

```

3.2 Instance metody (nestatické)

Nestatické metody vyžadují instanci třídy:

CS

```

1 class Kalkulacka {
2     public int Soucet(int a, int b) {
3         return a + b;
4     }
5 }
6
7 class Program {
8     static void Main() {
9         Kalkulacka calc = new Kalkulacka();
10        int vysledek = calc.Soucet(5, 3); // volání přes instanci
11    }
12 }

```

3.3 Různé způsoby volání

CS

```

1 class Demo {
2     static int Mocnina(int zaklad, int exponent) {
3         int vysledek = 1;
4         for (int i = 0; i < exponent; i++) {
5             vysledek *= zaklad;
6         }
7         return vysledek;
8     }
9
10    static void Main() {
11        // S proměnnými
12        int x = 2, y = 3;
13        int a = Mocnina(x, y);
14
15        // S konstantami
16        int b = Mocnina(5, 2);
17
18        // S výrazem
19        int c = Mocnina(3, 1 + 1);
20
21        // Vnořené volání
22        int d = Mocnina(Mocnina(2, 2), 2); // (2^2)^2 = 16
23
24        // Výsledek jako argument
25        Console.WriteLine(Mocnina(2, 5));
26    }
27 }

```

4 Formální a skutečné parametry

4.1 Formální parametry

- Parametry v **definici** metody
- Definují typ a název vstupních hodnot
- Fungují jako lokální proměnné uvnitř metody
- Existují pouze během provádění metody

4.2 Skutečné parametry (argumenty)

- Hodnoty předané při **volání** metody
- Mohou být: literály, proměnné, výrazy
- Musí odpovídat typem a počtem formálním parametrům
- Pořadí musí souhlasit (nebo použít pojmenované argumenty)

CS

```

1 static double VypocetCeny(double cena, double dph, double sleva) {
2     // cena, dph, sleva jsou FORMÁLNÍ parametry
3     return cena * (1 + dph) * (1 - sleva);
4 }
5
6 static void Main() {
7     double zakladniCena = 1000;
8     double sazba = 0.21;
9     double slevaProcent = 0.1;
10
11     // zakladniCena, sazba, slevaProcent jsou SKUTEČNÉ parametry
12     double konecnaCena = VypocetCeny(zakladniCena, sazba, slevaProcent);
13
14     // Také lze použít literály jako skutečné parametry
15     double cena2 = VypocetCeny(500, 0.21, 0.05);
16 }

```

4.3 Pojmenované argumenty

C# umožňuje předávat argumenty podle názvu:

CS

```

1 static void VytvorUzivatele(string jmeno, int vek, string mesto) {
2     Console.WriteLine($"{jmeno}, {vek} let, {mesto}");
3 }
4
5 static void Main() {
6     // Standardní pořadí
7     VytvorUzivatele("Jan", 25, "Praha");
8
9     // Pojmenované argumenty (libovolné pořadí)
10    VytvorUzivatele(mesto: "Brno", jmeno: "Petra", vek: 30);
11
12    // Kombinace
13    VytvorUzivatele("Karel", vek: 28, mesto: "Ostrava");
14 }

```

4.4 Volitelné parametry

Parametry mohou mít výchozí hodnoty:

CS

```
1 static void Pozdrav(string jmeno, string zprava = "Ahoj") {
2     Console.WriteLine($"{zprava}, {jmeno}!");
3 }
4
5 static void Main() {
6     Pozdrav("Jan"); // použije výchozí "Ahoj"
7     Pozdrav("Petra", "Dobrý den"); // použije "Dobrý den"
8 }
```

5 Modifikátory parametrů

C# nabízí různé způsoby předávání parametrů pomocí modifikátorů.

5.1 Předávání hodnotou (výchozí)

- Standardní způsob bez modifikátoru
- Vytvoří se **kopie** hodnoty
- Změny uvnitř metody **neovlivní** původní proměnnou
- Vhodné pro hodnotové typy (int, double, bool...)

CS

```
1 static void Zmena(int x) {
2     x = 100; // změní pouze lokální kopii
3     Console.WriteLine($"Uvnitř metody: {x}");
4 }
5
6 static void Main() {
7     int a = 5;
8     Zmena(a);
9     Console.WriteLine($"Po volání: {a}"); // stále 5
10 }
```

5.2 Modifikátor ref

- Předává **referenci** na původní proměnnou
- Změny uvnitř metody **ovlivní** původní proměnnou
- Proměnná **musí být inicializována** před voláním
- Při volání musí být použito klíčové slovo **ref**

CS

```
1 static void Zmena(ref int x) {
2     x = 100; // změní původní proměnnou
3 }
4
5 static void Prohod(ref int a, ref int b) {
6     int temp = a;
7     a = b;
8     b = temp;
9 }
10
```

```

11 static void Main() {
12     int cislo = 5;
13     Zmena(ref cislo);
14     Console.WriteLine(cislo); // vypíše 100
15
16     int x = 10, y = 20;
17     Prohod(ref x, ref y);
18     Console.WriteLine($"{x}, {y}"); // vypíše 20, 10
19 }

```

5.3 Modifikátor out

- Podobný jako `ref`, ale proměnná **nemusí být inicializována**
- Metoda **musí přiřadit** hodnotu parametru před ukončením
- Vhodné pro vracení více hodnot z metody
- Při volání musí být použito klíčové slovo `out`

CS

```

1 static bool TryParse(string text, out int vysledek) {
2     try {
3         vysledek = int.Parse(text);
4         return true;
5     } catch {
6         vysledek = 0;
7         return false;
8     }
9 }
10
11 static void VratViceHodnot(int x, out int dvojnásobek, out int trojnásobek) {
12     dvojnásobek = x * 2;
13     trojnásobek = x * 3;
14 }
15
16 static void Main() {
17     // out - nemusí být inicializováno
18     int hodnota;
19     if (TryParse("123", out hodnota)) {
20         Console.WriteLine($"Převod OK: {hodnota}");
21     }
22
23     // C# 7.0+ inline deklarace
24     if (TryParse("456", out int cislo)) {
25         Console.WriteLine($"Převod OK: {cislo}");
26     }
27
28     // Vracení více hodnot
29     VratViceHodnot(5, out int dva, out int tri);
30     Console.WriteLine($"2x: {dva}, 3x: {tri}"); // 10, 15
31 }

```

5.4 Modifikátor in (C# 7.2+)

- Předává referenci, ale **pouze pro čtení**
- Optimalizace pro velké struktury

- Zabrání změně hodnoty uvnitř metody
- Volitelné použití klíčového slova při volání

cs

```
1 static double VypocetVzdalenosti(in Vector3 bod1, in Vector3 bod2) {
2     // bod1 a bod2 nelze měnit
3     double dx = bod2.X - bod1.X;
4     double dy = bod2.Y - bod1.Y;
5     double dz = bod2.Z - bod1.Z;
6     return Math.Sqrt(dx * dx + dy * dy + dz * dz);
7 }
```

5.5 Modifikátor params

- Umožňuje předat **proměnný počet** argumentů
- Argumenty jsou předány jako pole
- Musí být poslední parametr v seznamu
- Lze volat bez argumentů (prázdné pole)

cs

```
1 static int Soucet(params int[] cisla) {
2     int vysledek = 0;
3     foreach (int cislo in cisla) {
4         vysledek += cislo;
5     }
6     return vysledek;
7 }
8
9 static void Main() {
10     Console.WriteLine(Soucet());           // 0
11     Console.WriteLine(Soucet(5));          // 5
12     Console.WriteLine(Soucet(1, 2, 3));    // 6
13     Console.WriteLine(Soucet(10, 20, 30, 40, 50)); // 150
14
15     // Nebo předat pole přímo
16     int[] pole = { 1, 2, 3, 4 };
17     Console.WriteLine(Soucet(pole));       // 10
18 }
```

5.6 Porovnání modifikátorů

Modifikátor	Inicializace před	Přiřazení v metodě	Může měnit
(žádný)	Ne	Ne	Ne (kopie)
ref	Ano	Ne	Ano
out	Ne	Ano (povinné)	Ano
in	Ano	Ne	Ne (readonly)
params	Ne	Ne	Ano (prvky pole)

6 Přetěžování metod (Method Overloading)

Přetěžování metod umožňuje definovat více metod se **stejným názvem**, ale s **různými parametry**.

6.1 Pravidla přetěžování

Metody se mohou lišit:

1. Počtem parametrů
2. Typy parametrů
3. Pořadím typů parametrů
4. Modifikátory parametrů (ref, out, in)

Metody se **neliší** pouze:

- Návratovým typem
- Názvy parametrů
- Modifikátory přístupu

6.2 Příklady přetěžování

cs

```

1 class Kalkulacka {
2     // Různý počet parametrů
3     public int Soucet(int a, int b) {
4         return a + b;
5     }
6
7     public int Soucet(int a, int b, int c) {
8         return a + b + c;
9     }
10
11     public int Soucet(int a, int b, int c, int d) {
12         return a + b + c + d;
13     }
14
15     // Různé typy parametrů
16     public double Soucet(double a, double b) {
17         return a + b;
18     }
19
20     public string Soucet(string a, string b) {
21         return a + b; // spojení řetězců
22     }
23
24     // Různé pořadí typů
25     public void Vypis(int cislo, string text) {
26         Console.WriteLine($"Číslo: {cislo}, Text: {text}");
27     }
28
29     public void Vypis(string text, int cislo) {
30         Console.WriteLine($"Text: {text}, Číslo: {cislo}");
31     }

```

32 }

6.3 Přetěžování s modifikátory

CS

```

1 class Demo {
2     // Standardní předání hodnotou
3     public void Metoda(int x) {
4         Console.WriteLine("Předání hodnotou");
5     }
6
7     // Přetížení s ref
8     public void Metoda(ref int x) {
9         Console.WriteLine("Předání referencí (ref)");
10    }
11
12    // Přetížení s out
13    public void Metoda(out int x) {
14        x = 10;
15        Console.WriteLine("Předání výstupem (out)");
16    }
17 }

```

6.4 Praktický příklad

CS

```

1 class FormatovaniTextu {
2     // Základní metoda
3     public string Formatuj(string text) {
4         return text.ToUpper();
5     }
6
7     // S délkou
8     public string Formatuj(string text, int maxDelka) {
9         string vysledek = text.ToUpper();
10        if (vysledek.Length > maxDelka) {
11            vysledek = vysledek.Substring(0, maxDelka) + "...";
12        }
13        return vysledek;
14    }
15
16    // S délkou a prefixem
17    public string Formatuj(string text, int maxDelka, string prefix) {
18        string vysledek = text.ToUpper();
19        if (vysledek.Length > maxDelka) {
20            vysledek = vysledek.Substring(0, maxDelka) + "...";
21        }
22        return prefix + vysledek;
23    }
24
25    static void Main() {
26        var fmt = new FormatovaniTextu();
27
28        Console.WriteLine(fmt.Formatuj("ahoj"));           // AH0J
29        Console.WriteLine(fmt.Formatuj("ahoj světe", 6));   // AH0J S...

```

```
30         Console.WriteLine(fmt.Formatuj("ahoj", 10, ">>> ")); // >>> AHOJ
31     }
32 }
```

6.5 Výhody přetěžování

1. **Flexibilita** - jedna metoda pro různé situace
2. **Čitelnost** - není nutné pamatovat různé názvy
3. **Intuitivnost** - programátor může volat stejnou metodu různými způsoby
4. **Konzistence** - jednotné rozhraní pro podobné operace
5. **Polymorfismus** - základ OOP v C#

Odpovědi na zadané podotázky

Každá otázka na začátku odpovědi obsahuje odkaz na konkrétní kapitulu v zápise s proklikem

Další materiály a zdroje

Žádné další zdroje
